Quiz 10: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 10 is Thursday Nov 5, at the end of class. About 15 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

> ❗ There is no coding section
>
> Part D is **short answer**: one or two sentences explaining why something behaves the way it does. The lab is tested by what it showed, not by writing the functions again.

Scope

Two lectures, and Lab 9.

The rest of the transformers lecture. Quiz 9 stopped at multi-head attention, which is where the Oct 22 class stopped. This quiz starts with what that deck still had: how position is actually encoded, the block around attention, LayerNorm, and encoder versus decoder. Query, key, value, the scaled dot product, multi-head, and the two masks were Quiz 9. You still need them. They are not retaught here. The Quiz 9 study guide is the reference for that half.

The Oct 29 lecture, large language models, and Lab 9. An LLM is the decoder from that deck, downloaded rather than trained. The new material is what changed so you can run one, and what you do with it: position as a rotation, memory, where it runs, temperature, embeddings and ingestion, a small fine-tune, and an orchestrator that listens for a tool call.

Not on the quiz. The sinusoidal formula does not need to be reproduced from memory; the sheet has it. Also off: the RoPE matrix, the RMSNorm formula, the SwiGLU formula, grouped-query algebra, Chinchilla, distributed training, and training either model. Lab 9 does not train anything.

Part 1: Position, after the embedding

Quiz 9 left position as a second vector added to the token. Two ways to build that vector, and one thing both of them are trying to do.

Sinusoidal positional encoding fills the vector with sines and cosines. k is the position, i indexes a pair of dimensions, d is the width, and $n = 10{,}000$:

$$P(k, 2i) = \sin\left(\frac{k}{n^{2i/d}}\right), \qquad P(k, 2i+1) = \cos\left(\frac{k}{n^{2i/d}}\right)$$

Each dimension is a wave of a different wavelength, so every position gets a different pattern. The useful part is the similarity. Neighbouring positions sit at cosine 0.97, and positions 20 apart sit at 0.61, and those two numbers do not depend on where in the sequence the pair sits. The encoding knows the gap.

Learned positional encoding is a table with one row per position, trained like any other parameter. BERT, GPT, and ViT use it. The table has a fixed number of rows. A model trained at length 2,048 has no row for index 2,049. Handing it a longer prompt does not work, because that slot was never trained. The sinusoid is defined for any k, so it does not have that hole. Week 10's rotation is the fix current models actually use.

Part 2: The block

Attention is one sublayer. A transformer block has two, and each is wrapped the same way:

$$\mathbf{x} \leftarrow \mathrm{LayerNorm}(\mathbf{x} + \mathrm{Attention}(\mathbf{x})), \qquad \mathbf{x} \leftarrow \mathrm{LayerNorm}(\mathbf{x} + \mathrm{FFN}(\mathbf{x}))$$

The residual is the same skip connection as in a residual network. The gradient has a path whose derivative is 1, so a stack of blocks can still train.

Attention mixes across positions. The feed-forward network does not. It is a two-layer MLP applied to each position on its own, usually widening to $4d$ and back. In one block the feed-forward weights are the larger share of the parameters.

LayerNorm normalizes a token using that token's own mean and standard deviation, then scales and shifts with learned parameters. **BatchNorm** normalizes it using the mean and standard deviation of the batch, and the padding zeros are part of that batch.

LayerNorm of a token is not a function of the padding or the other tokens, so a token that is held fixed does not move when the other rows become padding. In the lecture figure that drift is 0.00. BatchNorm is a function of those rows, so the same token moves by 2.20.

Part 3: Encoder, decoder, or both

A **decoder block** is an encoder block plus two changes. Its self-attention is causally masked, so position t sees only positions up to t. It adds a **cross-attention** sublayer between the self-attention and the feed-forward network.

In self-attention, the queries, keys, and values all come from the same sequence. In cross-attention, the queries come from the decoder and the keys and values come from the encoder. Each decoder position can read the whole input. That is what replaced squeezing a source sentence into one vector.

Which half you keep decides the model:

architecture	what it sees	example	what it is for
encoder only	the whole input at once	BERT	classification, tagging, extractive question answering
decoder only	everything before this token	GPT	generating text
encoder-decoder	the input fully, the output causally	T5	translation, summarization

Bidirectional context is for understanding. Causal masking is for generating. You cannot generate with bidirectional attention, because the token you would attend to does not exist yet.

Training and generation are not the same pass. Training feeds the true sequence and scores every next token at once. The causal mask is what stops each position from reading its own target. Generation does not have the next token yet, so it is one forward pass per new token. An LLM is this decoder, at a size you download.

Part 4: Three training runs

The same weights go through three runs, in this order.

Pretraining predicts the next token on a very large amount of text. It is the only expensive run, and it is where language and facts come from.

Supervised fine-tuning shows the model examples of the answer you wanted. It starts answering the question instead of continuing the prompt.

Preference optimization pushes the model toward the answer a person ranked higher. RLHF learns a reward from those rankings and then trains against the reward. DPO skips the reward model and trains on the pairs directly.

The second and third runs are small. They change behavior. A preference run cannot add a fact that was never in the pretraining text.

The public models from 2017 through 2023 are that same architecture, made larger. An 8 billion parameter model is something you can run. A frontier model is something you call. You do not train either one for this course.

Part 5: Rotary position embedding

Absolute position was a vector for a slot. **Rotary position embedding stores the position as a rotation.** The same word is the same vector, turned by its position times one base angle. In the lecture figure, "dog" at position 1 is rotated by θ, at position 2 by 2θ, and at position 4 by 4θ.

The query and the key are both rotated. Their dot product then depends on the gap between them, not on the absolute index. A gap the model has seen still works further along in a longer prompt. That is why the context window can extend past the training length. It is a setting on the model you run.

Three other names are enough to read a current model card. You do not need their formulas.

RMSNorm is the normalization current models use. It rescales a vector and does not subtract the mean. It replaced LayerNorm because it is cheaper at this size.

Grouped-query attention shares one set of keys and values across several query heads. Fewer keys and values means a smaller cache, which is what makes a long context fit in memory.

SwiGLU is the activation inside the feed-forward layer. It replaced the two-layer network from the block above.

Part 6: Running one

The number in the name, 8B or 70B, is the parameter count. Whether it runs is the memory.

At 16-bit precision each parameter is 2 bytes, so the weights of an 8 billion parameter model are about 16 GB. **Quantization** stores each weight in fewer bits. At 4 bits those weights are about 4 GB, which is small enough to load on a laptop. The network is the same shape. It is a little less accurate.

The context window is counted in tokens. In English a token is roughly three quarters of a word, so a 4,096-token window is about 3,000 words.

The weights live in one of three places.

- **Your machine**, with Ollama or LM Studio, when the documents cannot leave. Hugging Face is the path when you need the tensors: the probabilities, an embedding, or a fine-tune. Ollama and LM Studio return an answer.
- **A server you control**, one shared GPU, documents stay, you pay for the machine.
- **An API**, a frontier model, the prompt leaves, you pay per token written.

Hosting is that choice: where the weights sit, and who pays for each generated token.

Part 7: Temperature, prefill, and the cache

The model returns a distribution. One completion is one draw from it.

Temperature divides the scores before the softmax. Low temperature makes the top token nearly certain. High temperature flattens the distribution, so unlikely tokens get drawn. **Top-p** keeps the smallest set of tokens whose probabilities add up to p, and samples inside that set.

Prefill reads the prompt in one forward pass. **Decode** writes the answer one token at a time, and each of those passes reads the weights. A long answer costs more than a long prompt. An API bill follows the tokens the model writes.

The **KV cache** stores the keys and values of tokens already seen. The next token reads them instead of recomputing the prefix, so the time for one more token stays flat. The cost moves to memory. The cache grows with the conversation, and a long chat can run out of GPU memory even when the weights themselves fit.

With temperature above zero, two runs of the same prompt can differ. Fix the temperature, save the prompt, and read several samples. If the bad answer is in every draw, the problem is the model or the prompt. If it shows up once, it is the sample.

Part 8: Embeddings and ingestion

An embedding model is a transformer that turns a piece of text into one vector. It is not the chat model.

Search embeds a question and compares it with vectors that were already stored. The nearest vector is the result. In the lecture sketch, "Will it rain in Paris?" sits near "Bring a coat in Paris," "Rain likely in Paris tomorrow," and the other weather lines, and far from "How to train a dog" and "RoPE rotates the query." That picture is a sketch, not a measured projection. Lab 9 measures a real embedding model. There, "Rain likely in Paris tomorrow" is nearest and "How to train a dog" is last. "Bring a coat in Paris" is in the neighborhood and is not the closest, because the question asked about rain.

Ingestion is preparing text so it can be searched later. You split a document into chunks, embed each chunk with that same model, and store the vectors. The question is embedded only when someone asks it. No weights change in either step.

Part 9: LoRA

Parameter-efficient fine-tuning trains a small correction and leaves the pretrained weights fixed. **LoRA** is the usual version.

W is a frozen matrix of shape $d \times d$. The forward pass also sends x through a thin path: A has shape $r \times d$, B has shape $d \times r$, and the output is $Wx + BAx$. The rank r is small. Often $r = 8$.

For one 4096×4096 matrix,

$$\frac{2 \cdot 4096 \cdot 8}{4096^2} = 0.00390625,$$

which is 0.39% of W. You save A and B, not a second copy of W. That percentage is one matrix, not the whole model.

Use LoRA when the failure is behavior: a format, a tone, a domain vocabulary. A missing fact belongs in the prompt, or in retrieval. A small fine-tune will not reliably add it.

Part 10: Agents

An agent is a language model plus an orchestrator. The orchestrator is a program outside the model. It reads the text the model writes and runs a tool only when that text matches a tool call. The format of the call is fixed in advance.

In the lecture figure the model writes "I should check the forecast" and then the line `tool: weather(Paris, tomorrow)`. Only the `tool:` line starts the orchestrator. The sentence above it does not. The tool returns `8°C, rain`, the orchestrator appends that text to the prompt, and the model then writes "Yes. Bring a coat." Another `tool:` line starts the orchestrator again.

Lab 9 uses the same rule on a shorter line, `tool: weather(Paris)`, and appends `result: 8°C, rain`. A paragraph with no such line is returned unchanged.

Chain of thought is a prompt that asks the model to write the steps before the final answer. The weights stay the same. Each written step is another forward pass, so a hard question gets more computation. On the Kojima et al. (2022) table, one model, text-davinci-002, the phrase "Let's think step by step" moves MultiArith from about 18 to about 79. On a one-step question it barely moves. The paired numbers in each cell are two ways of reading the final answer out of the generated text, not two models.

Part 11: Lab 9

Lab 9 runs DistilGPT-2, about 82 million parameters, and a separate embedding model. There is no training. Sampling is on CPU with seed 6600, so a GPU and a laptop agree.

The prompt is "The Eiffel Tower is in the city of".

- The top token is **London**, probability about **0.111**. Paris is on the list and is not first. The forward pass succeeded. The fact is wrong. The lecture chart was a larger GPT-2, and Paris led there.
- Twenty draws at temperature **0.5** use **4** different tokens, and London is most of them. Twenty draws at temperature **1.5** use **20** different tokens. Same prompt, same weights, same seed.

- Six chunks are embedded and stored. The question "Will it rain in Paris?" ranks **"Rain likely in Paris tomorrow"** first and **"How to train a dog"** last.
- `tool: weather(Paris)` appends `result: 8°C, rain`. The sentence "I should check the forecast" does not run the tool.
- For $d = 4096$ and $r = 8$, the frozen matrix has 16,777,216 weights and the two small matrices have 65,536. That is **0.39% of** W.

How to prepare

1. **Say what a learned position table cannot do**, and what the sinusoid's similarity depends on. The formula can stay on the sheet. The gap is the point.
2. **Say why LayerNorm does not move when the other rows become padding**, and why BatchNorm does. The number in the figure is 2.20.
3. **Name the three architectures** and what each one is allowed to see. Cross-attention is the one where the queries and the keys come from different sequences.
4. **Separate the three training runs.** Only pretraining adds facts.
5. **Say what RoPE rotates, and why the score depends on the gap.** Same word, both the query and the key.
6. **From Lab 9**, say why London can outrank Paris and still be a successful forward pass, why twenty draws at temperature 1.5 are not twenty bugs, and why the dog sentence is last.
7. **Say which line runs a tool**, and what the 0.39% is a fraction of. One matrix, not the whole model.

The formula sheet has the sinusoidal positional formula, the block in both orders, LayerNorm and BatchNorm, the temperature softmax, the weight-memory product, and the LoRA shapes and counts. It does not say what any of them are for.